

Week 4: Stochastic Optimal Control

The general theory behind the Avellaneda-Stoikov worked example

North Carolina State University | Summer 2026

NC STATE UNIVERSITY

Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

- ▶ Apply the dynamic programming principle in continuous time
- ▶ Understand the HJB equation and the verification theorem
- ▶ See why CARA / CRRA utilities give tractable problems

- ▶ Controlled diffusions and admissible controls
- ▶ The dynamic programming principle (Bellman)
- ▶ The infinitesimal generator and the HJB equation

- ▶ Verification theorem: a smooth HJB solution IS the value function
- ▶ CARA vs CRRA: where wealth enters the optimal policy
- ▶ Why market making uses CARA

The Stochastic-Control Pipeline



Verification theorem: a smooth HJB solution IS the value function

$$V(t, x) = \sup_u \mathbb{E} \left[\int_t^T f(s, X_s, u_s) ds + g(X_T) \right]$$

Why: The value function is the best achievable expected reward from (t, x) onward. We define the whole problem through V because Bellman's principle then gives a local (differential) equation instead of an intractable global optimization.

$$V(t, x) = \sup_u \mathbb{E}[V(t+dt, X_{t+dt}) + f dt]$$

Why: The dynamic programming principle: an optimal policy is optimal from every later state too. Expanding $V(t + dt, \cdot)$ with Ito's lemma turns this into the HJB PDE.

$$\partial_t V + \sup_u \{ \mathcal{L}^u V + f(t, x, u) \} = 0, \quad V(T, x) = g(x)$$

Why: The HJB equation. $\mathcal{L}^u$ is the generator of the controlled diffusion; the $\sup$ is the instantaneous optimal control. This is the engine behind every model in the course – Avellaneda-Stoikov is one special case.

CARA: $U(w) = -e^{-\gamma w} \Rightarrow$ policy independent of w

Why: Constant absolute risk aversion makes marginal utility scale-free in wealth, so the optimal quotes do not move with the cash balance. CRRA ($U = w^{1-\gamma}/(1-\gamma)$) instead makes the policy scale WITH wealth – natural for investment, awkward for a desk.

- ▶ Theory week – no Coincall data required.
- ▶ Optional: estimate drift/vol from perp klines to parameterize Merton.

Cross-check exact paths at <https://docs.coincall.com/>

```

import torch
# Merton under CARA, solved by backward value iteration on a wealth grid.
w = torch.linspace(-5, 5, 401); dt = 1e-3; mu, r, sig, gam = 0.08, 0.02, 0.2, 1.0
V = -torch.exp(-gam * w) # terminal utility
for _ in range(1000): # march backward in time
    # optimal risky holding  $\pi^* = (\mu - r)/(\text{gam} \cdot \text{sig}^2)$  is wealth-independent (CARA)
    pi = (mu - r) / (gam * sig**2)
    drift = (r*w + pi*(mu - r))
    diff = 0.5 * (pi*sig)**2
    dV = torch.gradient(V, spacing=(w[1]-w[0]).item())[0]
    d2V = torch.gradient(dV, spacing=(w[1]-w[0]).item())[0]
    V = V + dt * (drift*dV + diff*d2V)
print("pi* (constant in wealth):", (mu - r)/(gam*sig**2))

```

- ▶ Pham (2009), Ch. 3-4; Cartea-Jaikumar-Penalva, Ch. 6